Reviewer Pack — Star Discrepancy of Clause-Vectors Lower-Bounds Resolution W…

Entry b22241903f08 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 23:43:12 UTC

Star Discrepancy of Clause-Vectors Lower-Bounds Resolution Width for Random 3-SAT

Entry ID: `b22241903f08` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-29 23:43:12 UTC

1. Conjecture

Field A (mathematical branch): Discrepancy theory (star discrepancy in the sense of Roth / Schmidt / Matousek) **Field B** (complexity object): Resolution refutation width w(F⊢⊥) on random unsatisfiable 3-CNFs

Statement:

> Let F be a random unsatisfiable 3-CNF over n variables at clause density 4.5. Let V_F ∈ [0,1]^{m×n} be the clause-variable matrix normalized coordinate-wise. Then E[D^*(V_F)] = Ω(log n) implies w(F⊢⊥) = Ω(n / log n), where D^* denotes the star discrepancy of the point set induced by the rows of V_F.

Rationale (proposer's reasoning):

> Star discrepancy measures how uniformly distributed a finite point set is in the unit cube, a central object in quasi-Monte Carlo and geometric discrepancy (Matousek 'Geometric Discrepancy'). High discrepancy implies clustering in projections, which we hypothesize corresponds to variable-heavy clauses that force wide resolution derivations. This connects dispersion in geometric space to proof complexity via variable scoping in refutations.

Taxonomy category: `DISPERSION_DISCREPANCY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `af78e45aafd6b5cf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Star discrepancy lower-bounds resolution width

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `discrepancy theory and resolution width` - `star discrepancy and clause variable matrix and random 3-SAT` - `geometric discrepancy and resolution refutation width and unsatisfiable 3-CNFs`

Top relevant hits considered: - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [http://arxiv.org/abs/chao-dyn/9311011v2] The Great Inequality In A Hamiltonian Planetary Theory - [http://arxiv.org/abs/0907.5244v1] IRSF/SIRIUS JHKs near-infrared variable star survey in the Magellanic Clouds - [http://arxiv.org/abs/1005.0979v1] Supersymmetry in Random Matrix Theory - [http://arxiv.org/abs/1908.05361v2] Placing quantified variants of 3-SAT and Not-All-Equal 3-SAT in the polynomial hierarchy - [http://arxiv.org/abs/1510.05486v2] Two types of branching programs with bounded repetition that cannot efficiently compute monotone 3-CNFs - [http://arxiv.org/abs/1203.3706v2] On the Complexity of Computing Minimal Unsatisfiable LTL formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import product

def generate_random_3cnf(n, m):
    literals = [f'x{i+1}', f'-x{i+1}' for i in range(n)]
    cnf = []
    for _ in range(m):
        clause = random.sample(literals, 3)
        cnf.append(' or '.join(clause))
    return ' and '.join(cnf)

def dpll_solve(cnf):
    def solve(assignment):
        if not cnf:
            return True
        literals = set()
        for clause in cnf:
            literals.update(set(clause.split()[:3]))
        literal = next(iter(literals))
        pos_literal, neg_literal = literal, f'-{literal}'
        if pos_literal in assignment and assignment[pos_literal]:
```

```
            return solve(assignment)
        elif neg_literal in assignment and not assignment[neg_literal]:
            return solve(assignment)
        else:
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if solve(new_assignment):
                return True
            new_assignment[literal] = False
            new_assignment[f'-{literal}'] = True
            if solve(new_assignment):
                return True
            return False
    return solve({})

def star_discrepancy(V_F, m, n):
    D = 0
    for a in product([0, 1], repeat=n):
        S = [sum(1 if V_F[i][j] >= a[j] else -1 for j in range(n)) for i in range(m)]
        D = max(D, abs(sum(S[i] * V_F[i][j] for i in range(m))) / math.sqrt(m))
    return D

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 30
    m = 135
    cnf = generate_random_3cnf(n, m)
    V_F = [[0.5 for _ in range(n)] for _ in range(m)]
    for i, clause in enumerate(cnf.split(' and ')):
        literals = clause.split(' or ')
        for literal in literals:
            if literal[0] == '-':
                var = int(literal[1:])
                V_F[i][var-1] = 1 - 1 / (2 * n)
            else:
                var = int(literal)
                V_F[i][var-1] = 1 / (2 * n)

    D_star = star_discrepancy(V_F, m, n)
    resolution_width = len(cnf.split(' and '))

    return {
        "metric_name": "resolution_width",
        "metric_value": resolution_width,
        "instances_tested": 1,
        "conjecture_holds": D_star >= math.log(n) * 2,  # Using a constant c=2 for simplicity
        "counterexample": "" if D_star >= math.log(n) * 2 else f"Star discrepancy {D_star} < {math
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(resu
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        counterexample = next(res["counterexample"] for res in results if res["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0eb35a8.py", line 87, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0eb35a8.py", line 71, in run_trial
    D_star = star_discrepancy(V_F, m, n)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0eb35a8.py", line 51, in star_discrepa
    D = max(D, abs(sum(S[i] * V_F[i][j] for i in range(m))) / math.sqrt(m))
                   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a0eb35a8.py", line 51, in <genexpr>
    D = max(D, abs(sum(S[i] * V_F[i][j] for i in range(m))) / math.sqrt(m))
                       ~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with IndexError before producing data; cannot evaluate support/falsification metrics | next: Debug the star_discrepancy function's index handling in test_a0eb35a8.py

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	cerebras	qwen-3-235b-a22b-ins	0	0	6359
2	preregistration	groq	llama-3.3-70b-versat	0	0	4145
3	novelty	groq	llama-3.3-70b-versat	0	0	4045
4	novelty	ollama_remote	qwen3:8b	0	0	26748
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13364
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11060
7	judge	ollama_remote	qwen3:8b	0	0	15407

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 81127 ms total latency. Provider mix: {'cerebras': 1, 'groq': 2, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/b22241903f08.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b22241903f08.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b22241903f08.tar.gz` (if generated)